

# Setting Up, and Learning to Look at Data

Dr. Naila Marir · College of Engineering, Effat University

Machine learning has a shape, and most of that shape is not the model. Today you will learn the shape, meet the tools, and get as far as a professional gets before writing a single line of modelling code: **you will know your data**. In Lab 2 you will train a model on it.

**The goal of this lab:** understand how a machine learning project works from end to end, and carry a real medical dataset through the first three stages of it.

Parts 1–3 are tools and vocabulary — short on purpose. **Part 4 is the lab**, and it is where you meet the data.

|                      |                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prerequisites</b> | A Google account. Python: variables, lists, functions, loops.                                                                                    |
| <b>Deliverable</b>   | The notebook with both <code># TODO</code> cells completed, renamed <code>MEAI502_Lab1_&lt;YourName&gt;.ipynb</code> and uploaded to Blackboard. |
| <b>Assessment</b>    | Exercise 60 %, reflection 40 %. See §5.                                                                                                          |
| <b>Notebook</b>      | <code>MEAI502_Lab1_Student.ipynb</code> — open it in Colab before you start reading.                                                             |

## 1 Getting Set Up

≈ 20 minutes

### 1.1 Google Colab

Colab is a Python notebook that runs on Google's computers, in your browser. Nothing to install.

1. Go to [colab.research.google.com](https://colab.research.google.com) and sign in.
2. File → New notebook, then click the title to rename it.
3. It saves itself to Google Drive, in a folder called **Colab Notebooks**.

A notebook is a list of **cells**: *code cells* run Python, *text cells* hold Markdown. Hover between two cells to get the **+ Code** / **+ Text** buttons. `Shift + Enter` runs a cell and moves on — that is the only shortcut you need today.

#### ⚠ THE ONE TRAP TO KNOW ABOUT

A notebook remembers **values**, not the order you see on screen. Set `x = 10`, run the cell below it, then go back, change `x` to 99 and run *only* that cell. The second cell's answer is now wrong, and nothing warns you. The numbers `[1]` `[2]` `[3]` on the left tell you the order things *actually* ran in.

**Before you submit anything:** Runtime → Restart and run all. If it does not survive that, it is not a result.

## 1.2 The runtime

The **runtime** is the machine Google lends you. Your variables, packages and files live there.

- Runtime → Change runtime type → **CPU** is all you need until deep learning (MEAI 505).
- Runtime → Restart session clears every variable.
- The free tier disconnects after about **90 minutes idle**.
- Anything you `!pip install` is **gone next session** — keep the install line in the notebook.

Everything this lab needs is already installed.

## 1.3 VS Code — for later, not for today

Colab is where you explore. **VS Code** is a code editor on your own laptop, and it is where a project with many files, a debugger and Git eventually lives. You will need it for your applied project. Set it up once, at home:

1. Install **Python 3.11+** from [python.org](https://python.org) and **VS Code** from [code.visualstudio.com](https://code.visualstudio.com).
2. In VS Code, install the **Python** and **Jupyter** extensions (Microsoft).
3. Give the project its own environment:

```
mkdir meai502 && cd meai502
python -m venv .venv           # a private Python just for this project
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
```

1. File → Open Folder... → meai502, drop the `.ipynb` inside, open it, then **Select Kernel → Python Environments → .venv**.

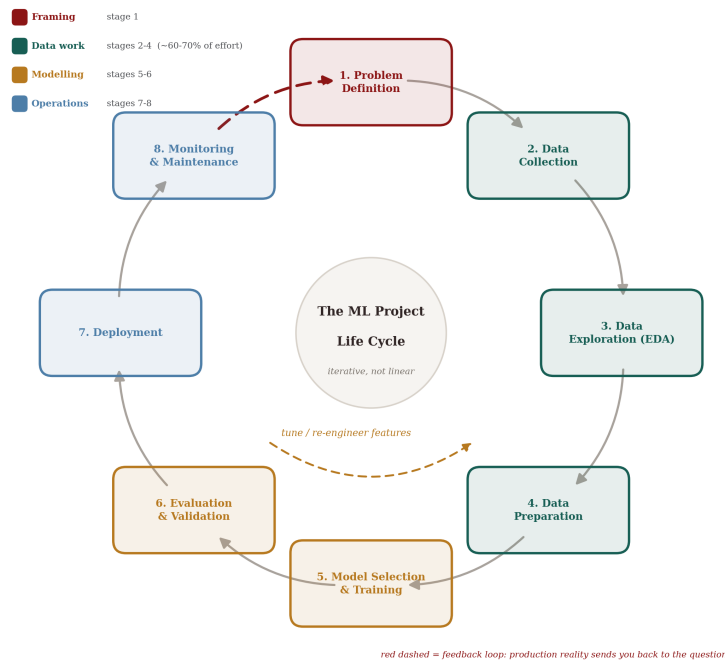
### WHY AN ENVIRONMENT PER PROJECT?

One project needs scikit-learn 1.3, another needs 1.8. A `venv` keeps them from fighting.

## 2 How an ML Project Works

≈ 20 minutes

A machine learning project is **not** "load data, call `.fit()`, publish". It is a cycle of eight stages, and the last one sends you back to the first.



**Figure 1 — The machine learning project life cycle.** Grey arrows are the normal flow; the amber dashed arrow is the tuning loop; the red dashed arrow is the feedback from production back to the original question.

### 2.1 The eight stages

| # | Stage                     | What it is for                                                            | Classic mistake                                    |
|---|---------------------------|---------------------------------------------------------------------------|----------------------------------------------------|
| 1 | <b>Problem definition</b> | Turn a vague need into a precise task with a measurable notion of success | Optimising accuracy when the decision needs recall |
| 2 | <b>Data collection</b>    | Get data that represents the population the model will actually serve     | A convenience sample from one site or device       |
| 3 | <b>Data exploration</b>   | Understand quality and relationships <i>before</i> modelling              | Going straight to <code>.fit()</code>              |
| 4 | <b>Data preparation</b>   | Build a clean, leak-free feature matrix                                   | Scaling before splitting                           |
| 5 | <b>Model training</b>     | Learn the mapping — and beat a baseline                                   | No baseline, so no idea if it is any good          |
| 6 | <b>Evaluation</b>         | Estimate honestly how it behaves on unseen data                           | One accuracy number on imbalanced data             |
| 7 | <b>Deployment</b>         | Put it where the decision is actually made                                | Preprocessing differs in production                |
| 8 | <b>Monitoring</b>         | Keep it correct while the world changes                                   | "Fire and forget"                                  |

- **It is a loop, not a line.** You will go back. Going back is the method, not a failure.

- **The data stages are the expensive ones.** Stages 2–4 are roughly **60–70 %** of the real effort. Modelling is the small, glamorous part.

**Today: stages 1, 2 and 3. Lab 2: stages 4, 5 and 6.** Stages 7 and 8 come later in the programme.

### 3 The Five Libraries

≈ 15 minutes

Five libraries do almost all the work. In the notebook you meet them one at a time, one cell each.

| Library             | Mental model                                                 | One line of it                         |
|---------------------|--------------------------------------------------------------|----------------------------------------|
| <b>NumPy</b>        | A fast grid of numbers you can do arithmetic on all at once  | <code>a * 2</code>                     |
| <b>pandas</b>       | A spreadsheet you drive with code ( <code>DataFrame</code> ) | <code>df["score"].isna().sum()</code>  |
| <b>Matplotlib</b>   | A canvas where every element is an object you can change     | <code>ax.plot(x, y)</code>             |
| <b>seaborn</b>      | Matplotlib with opinions; it speaks <code>DataFrames</code>  | <code>sns.barplot(data=df, ...)</code> |
| <b>scikit-learn</b> | One interface for every algorithm                            | <code>model.fit(X, y)</code>           |

The scikit-learn pattern is worth memorising now, because it never changes — every model in the library, from linear regression to a neural network, has these two methods:

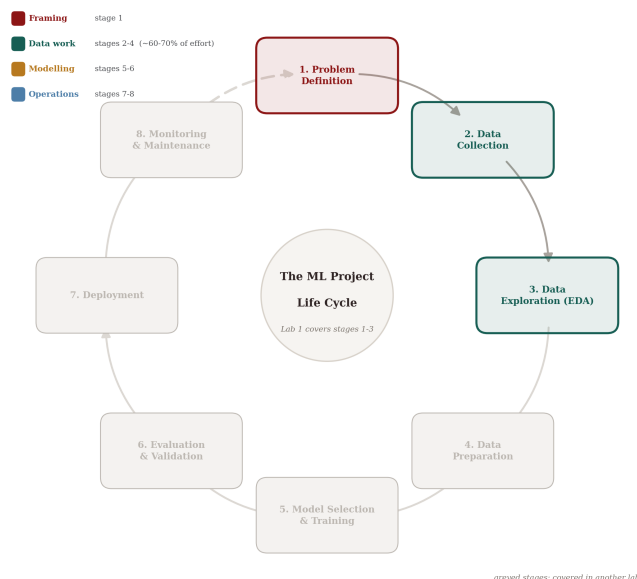
```
model.fit(X, y)      # learn from data
model.predict(X_new) # produce predictions
```

Set the plotting style once, at the top of the notebook. A course, a paper or a thesis should look like one document, not twenty:

```
PALETTE = ["#8C1515", "#4D7EA8", "#175E54", "#B7791F"]
sns.set_theme(style="whitegrid", palette=PALETTE)
RANDOM_STATE = 42          # one seed everywhere, so results repeat
```

## 4 Stages 1 to 3 on Real Data

≈ 50 minutes — this is the heart of the lab



**Figure 2 — Where you are.** Today covers stages 1 to 3. The greyed stages are Lab 2's, and later.

### THE DATASET

**Breast Cancer Wisconsin (Diagnostic).** 569 biopsies. For each one, 30 numeric measurements taken from a digitised image of a fine-needle aspirate — radius, texture, concavity and so on — and a diagnosis: **malignant** or **benign**. It ships inside scikit-learn, so there is nothing to download.

### STAGE 1

#### Problem Definition

**The need.** A pathology lab wants a second-opinion tool that flags biopsies likely to be malignant, so a radiologist reviews the risky ones first.

**ML task:** binary classification. **Target  $y$ :** 1 = malignant, 0 = benign. **Features  $x$ :** the 30 image measurements.

**The metric: recall on the malignant class** — of all the truly malignant cases, what fraction do we catch? We also report precision, so we know how many false alarms that costs.

**Why not accuracy?** Because the two mistakes are not equally bad. A false alarm costs one extra review. A missed malignancy can cost a life. The metric has to encode that asymmetry.

No code yet. Stage 1 is a thinking stage — and skipping it is how projects go wrong.

**STAGE 2****Data Collection**

A real project spends weeks here: getting access, checking consent, writing a data dictionary. Ours is one function call — but notice what we still check straight away.

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer(as_frame=True)
X = data.data           # the 30 measurements, as a DataFrame
y_raw = data.target     # sklearn encodes 0 = malignant, 1 = benign
X.shape                 # (569, 30)
```

**Re-encode the target.** scikit-learn ships this dataset with 0 = malignant. We want the opposite, because **the positive class should be the thing you are trying to detect** — every metric you compute is about class 1. Get this backwards and your recall will quietly measure the wrong thing.

```
y = (y_raw == 0).astype(int)    # now: 1 = malignant, 0 = benign
```

**Then look at the actual table** — not a summary of it, the rows themselves (`X.head()`). Thirty columns is too many to read at once, so they are grouped: the 30 come from **10 base measurements** of the cell nuclei, each reported three ways — the **mean**, the **error**, and the **worst** (largest) value in the image.

**STAGE 3****Data Exploration (EDA)**

Four questions, in this order.

**3.1 IS ANYTHING MISSING?**

```
X.isna().sum().sum()    # 0
```

Zero. That is a teaching dataset, not real life — expect to spend most of Stage 4 on exactly the problem this file does not have.

**3.2 WHAT DO THE NUMBERS LOOK LIKE?**

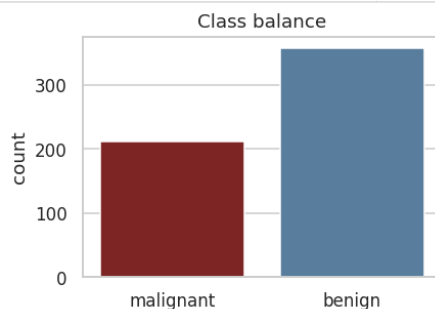
```
X[["mean radius", "mean texture", "mean area", "mean concavity"]].describe()
```

Notice the **scales**: mean area runs into the hundreds while mean concavity sits below 1. Some models do not care. Others care a great deal. That is a Stage 4 problem, and Lab 2 deals with it.

**3.3 IS THE TARGET BALANCED?**

```
output
benign      357
malignant   212

malignant rate: 37.3%
```



**Figure 3 — Class balance.** Moderately imbalanced.

### ⚠ DO THE ARITHMETIC

37.3 % malignant means 62.7 % benign. So a "model" that always answered *benign* would already be right 62.7 % of the time — while catching **zero** cancers.

That single number is the reason our metric is recall and not accuracy. Hold on to it: in Lab 2 it becomes the bar our real model has to clear.

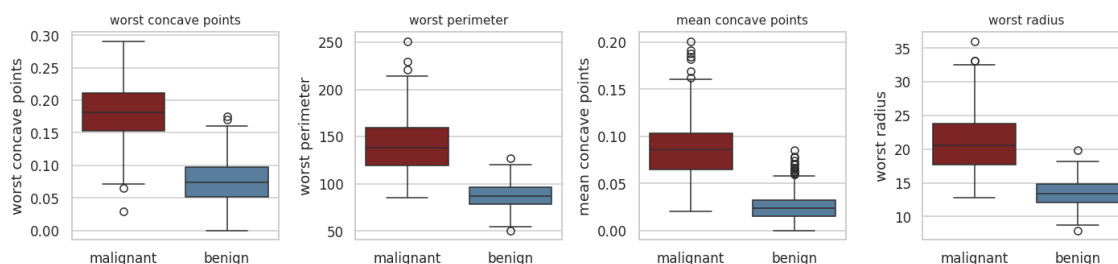
## 3.4 WHICH FEATURES RELATE TO THE DIAGNOSIS?

```
corr = X.corrwith(y).sort_values(ascending=False)
corr.head(5)
```

*output*

```
worst concave points    0.794
worst perimeter         0.783
mean concave points    0.777
worst radius            0.776
mean perimeter          0.743
```

All positive and all fairly strong — good news. It means the signal we need is present in the features, before we have trained anything. If every correlation had been near zero, no model would have saved us.



**Figure 4 — The four features most associated with malignancy, by diagnosis.** The box is the middle half of the data, the line inside it is the median, the dots are outliers. The two boxes barely overlap — that is what "this feature separates the classes" looks like. If they had overlapped completely, the feature would be useless.

## OUTPUT

### What Stage 3 told us

Write this down — it is the product of the stage, and it is what you hand to Stage 4:

1. **569 rows, 30 numeric features, no missing values.** Nothing to clean.

2. **The features are on wildly different scales** (area in the hundreds, concavity below 1). Some models will need that fixed.
3. **37.3 % malignant**. Moderately imbalanced — accuracy will flatter any model, so we use recall.
4. **Several features separate the classes clearly**. There is real signal here.

That is a complete Stage 3. Notice that we have still not trained anything — and we already know what the model will need, and how we will judge it.

## 5 Wrap-up

≈ 15 minutes

### 5.1 Exercise: explore two features of your own choosing (60 %)

In `TODO 1`, pick **two features that are not in the top four** — anything from `X.columns`. For each one:

1. Plot it by diagnosis (a boxplot, exactly as we did in §3.4).
2. Report its correlation with the target.
3. Write one sentence saying whether you would expect it to help a model, and why.

The marks are in step 3. A plot with no reading of it is worth very little.

### 5.2 Reflection: frame a problem from your own field (40 %)

Stage 1 only. In `TODO 2`, pick a problem you actually care about — health, energy, transport, education, finance, anything — and answer three questions, one or two sentences each:

| Question        | What to write                                                             |
|-----------------|---------------------------------------------------------------------------|
| <b>decision</b> | Who uses the prediction, and what do they do differently because of it?   |
| <b>target</b>   | What exactly is $y$ ? Name it, and say whether it is a class or a number. |
| <b>metric</b>   | How will you judge it, and why that metric for <i>this</i> decision?      |

Be concrete. "Predict student performance" is not a target variable. "High accuracy" is not a metric.

### 5.3 Before you submit

1. Runtime → Restart and run all — the whole notebook must run top to bottom without errors.
2. File → Download → `.ipynb`, rename it `MEAI502_Lab1_<YourName>.ipynb`.
3. Upload it to Blackboard.

### 5.4 Quick self-check

1. Why does the positive class have to be the thing you are trying to detect?
2. The dataset is 37.3 % malignant. What accuracy does a model get for free, and what does it catch?
3. Stage 3 found no missing values and no cleaning to do. Name two things it *did* find that will change what happens in Stage 4.

**ANSWERS**

1. Because recall, precision and F1 are all computed for class 1. If benign were class 1, "recall" would measure how many healthy patients you found — a number nobody asked for, reported under the name of one they did.
2. 62.7 % accuracy, by always answering benign, while catching zero malignant cases. That is the baseline Lab 2 starts from.
3. (i) The features are on wildly different scales, which will need fixing before a linear model can use them; (ii) the classes are imbalanced, which is why the metric is recall. Either of those, or: several features separate the classes well, so there is signal to find.

**5.5 Next lab: stages 4 to 6, your first model**

You now know this dataset. Lab 2 turns it into a model: **Stage 4** — split the data into a training set and a sealed test set, and fix those wild scales; **Stage 5** — train a baseline first, then a real model; **Stage 6** — measure it with the metric we chose today, and read a confusion matrix.

**Before Lab 2:** re-run the notebook with `Restart` and `run all`, and be ready to say what *recall* is and why we chose it.

**5.6 References**

1. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.), Ch. 2.
2. pandas — 10 minutes to pandas — [pandas.pydata.org/docs/user\\_guide/10min.html](https://pandas.pydata.org/docs/user_guide/10min.html)
3. seaborn tutorial — [seaborn.pydata.org/tutorial.html](https://seaborn.pydata.org/tutorial.html)
4. Google Colab — Overview of Colaboratory Features — [colab.research.google.com/notebooks/basic\\_features\\_overview.ipynb](https://colab.research.google.com/notebooks/basic_features_overview.ipynb)
5. Dua, D. & Graff, C. — UCI Machine Learning Repository, *Breast Cancer Wisconsin (Diagnostic)*.